Linked list. (Part 1)

Debarka Sengupta, Ph.D., FNASc.

Content

Arrays vs. Linked Lists
Heap vs. Stack
Application of Linked Lists
Stack and Heap Memory Allocation Examples
Struct vs. Class
malloc(), free() vs. new, delete()
Node
Linking nodes
Traversing the linked list with the help of recursion
Insertion and Deletion Operations in Linked Lists:
Exercise
Deep dive topics
References

Arrays vs. Linked Lists

Problems with Arrays:

- Prediction of size:
  - Specifying the exact size of an array upfront can be challenging in practical scenarios.
- Static memory allocation:
  - Memory allocation needs to be determined during compile time.
- Wastage of space:
  - Inefficient utilization of storage memory.
- Inefficient implementation of insertions and deletions:
  - Insertions and deletions often require costly shift operations.

Advantages of Linked Lists:

- Dynamic memory allocation:
  - Memory is allocated as needed.
- Easy implementation of insertion and deletion operations.
- Dynamic resizing:
  - Linked lists can grow or shrink during program execution.
- No need to specify size in advance.
- Support for stacks and queues.
- Flexibility in rearranging items efficiently.

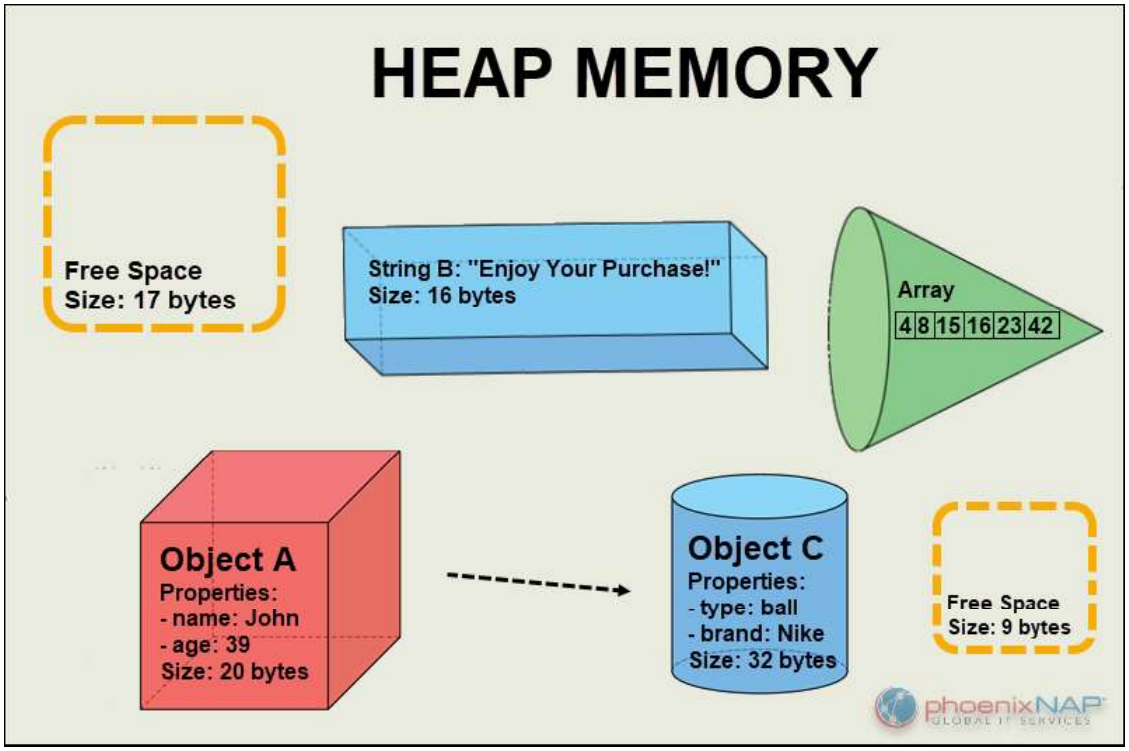
Disadvantages of Linked Lists:

- Wastage of memory:
  - Pointers require additional memory for storage.
- Linear access time:
  - Accessing elements requires sequential traversal, resulting in O(n) time complexity.
- Complex basic operations:
  - Certain operations, like finding the last node or determining insertion points, may necessitate iterating through the entire list.
- Difficulty in reverse traversing.

Heap vs. Stack

Heap Memory:

- Dynamic allocation:
  - Memory is allocated during runtime.
- Usage:
  - Used for objects and data structures whose size or lifetime cannot be determined at compile time.
- Management:
  - Managed manually or automatically (garbage collected in languages like Java).
- Access:
  - Accessible globally.
- Allocation:
  - Slower allocation and deallocation compared to stack memory.
- Lifetime:
  - Lifetime is not tied to the scope of a function.

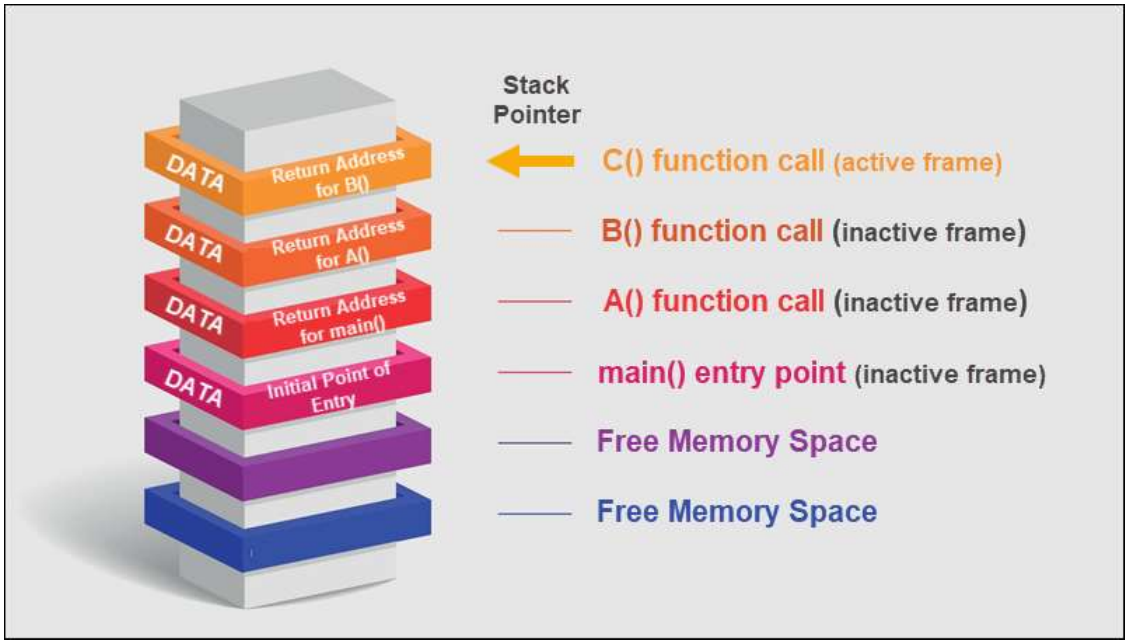


Heap memory.

Stack Memory:

- Static allocation:
  - Memory is allocated at compile time.
- Usage:
  - Used for function calls, local variables, and function parameters.
- Management:
  - Automatically managed by the compiler.
- Access:
  - Limited to the scope of the function.
- Allocation:
  - Faster allocation and deallocation compared to heap memory.
- Lifetime:
  - Lifetime is tied to the scope of a function; memory is deallocated when the function exits.

In summary, heap memory allows for dynamic allocation of memory at runtime with a longer lifetime, while stack memory is statically allocated with a shorter lifetime tied to function scopes.



Stack memory.

Application of Linked Lists

1. **Polynomial Manipulation representation**: Linked lists can represent polynomials efficiently, with each node storing a term (such as coefficient and exponent), enabling easy addition, subtraction, and multiplication of polynomials.
2. **Addition of long positive integers**: Linked lists can store digits of large integers, allowing for addition by performing digit-wise arithmetic and carrying over the excess to the next node.
3. **Representation of sparse matrices**: Sparse matrices, which have many zero elements, can be represented compactly using linked lists to store only the non-zero elements and their positions.
4. **Symbol table creation**: Linked lists can be used to create symbol tables in compilers or interpreters, where each node represents a symbol (such as variables or functions) and its associated information (like data type or memory location).
5. **Mailing list**: Linked lists can manage mailing lists efficiently, with each node representing a member and linking to the next member, facilitating operations like adding, removing, or iterating over the list.
6. **Memory management**: Linked lists are used in memory management systems of operating systems, with each node representing a block of memory and maintaining information about its status (free or allocated) and adjacent blocks.
7. **Linked allocation of files**: In file systems, linked allocation can be used where each node in a linked list represents a disk block, linking to the next block of a file, allowing for efficient storage and retrieval.
8. **Multiple precision arithmetic**: Linked lists can store digits of large numbers in multiple precision arithmetic, enabling arithmetic operations on numbers exceeding the capacity of standard data types.

Stack and Heap Memory Allocation Examples

C++ Pseudocode:

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Allocating memory for integer 'a' on the stack..." << endl;
    int a = 10; // stored in stack
    cout << "Memory for 'a' allocated on the stack successfully." << endl << endl;

    cout << "Allocating memory for an integer on the heap..." << endl;
    int* p = new int(); // allocate memory in heap
    cout << "Memory for the integer allocated on the heap successfully." << endl << endl;

    cout << "Assigning value '10' to the dynamically allocated memory..." << endl;
    *p = 10;
    cout << "Value assigned successfully." << endl << endl;

    cout << "Deallocating the dynamically allocated memory..." << endl;
    delete p;
    cout << "Memory deallocated successfully." << endl << endl;

    cout << "Allocating memory for an array of 4 integers on the heap..." << endl;
    p = new int[4]; // array in heap allocation
    cout << "Memory for the array allocated on the heap successfully." << endl << endl;

    cout << "Deallocating the memory for the array..." << endl;
    delete[] p;
    cout << "Memory deallocated successfully." << endl << endl;

    p = NULL; // free heap
    cout << "Memory freed successfully." << endl;

    return 0;
}
```

💡 **What do the first two lines of the above code signify?**

The line `#include <iostream>` is a preprocessor directive in C++ that tells the compiler to include the contents of the iostream header file in the program.

The `iostream` header file contains declarations for the standard input/output stream objects (`cin`, `cout`, `cerr`, `clog`) and input/output manipulators (such as `endl`).

The line `using namespace std;` is a declaration that indicates to the compiler that the program will be using entities from the `std` namespace. The `std` namespace contains all the standard C++ library components.

Combining both lines allows you to use the functionality provided by the `iostream` library, such as input and output operations, without having to prefix them with `std::`.

For example, instead of writing `std::cout`, you can simply write `cout` after including the `iostream` header and using the `std` namespace.

Struct vs. Class

Struct and class are pretty much the same, except that the members of a struct are public by default, while the members of a class are private by default. So, you'd pick whichever one made the coding task easy.

malloc(), free() vs. new, delete()

malloc/free:

- **C-style Memory Allocation:** `malloc` and `free` are functions provided by the C standard library for dynamic memory allocation and deallocation.
- **No Constructor/Destructor Invoked:** `malloc` allocates memory but does not call constructors for objects. It simply reserves a block of memory.
- **Low-Level Approach:** It operates at a low level, providing basic memory allocation functionality without any type checking.
- **Usage:** In C and in C++ when dealing with legacy code or interoperability with C libraries.

Example:

```
int* ptr = (int*)malloc(sizeof(int)); // Allocates memory for an integer
// Use the allocated memory
free(ptr); // Deallocates the memory
```

new/delete:

- **C++ Dynamic Memory Management:** `new` and `delete` are operators in C++ for dynamic memory allocation and deallocation.
- **Constructor/Destructor Called:** `new` not only allocates memory but also calls the constructor to initialize the object. `delete` calls the destructor before deallocating the memory.
- **Type Safety:** Since `new` is a C++ operator, it provides type safety and automatically determines the size of the object.
- **Usage:** Preferred in modern C++ for memory management, especially when dealing with objects.

Example:

```
int* ptr = new int; // Allocates memory for an integer and calls its constructor
// Use the allocated memory
delete ptr; // Calls the destructor and deallocates the memory
```

Key Differences:

1. **Type Safety:** `new`/`delete` are type-safe and work with C++ objects, while `malloc`/`free` are not type-safe and work with raw memory.
2. **Constructor/Destructor:** `new`/`delete` call the constructor/destructor of objects, whereas `malloc`/`free` do not.
3. **C++ vs. C:** `new`/`delete` are C++ operators, while `malloc`/`free` are C functions, commonly used in C programming.
4. **Size Calculation:** `malloc` requires the programmer to manually calculate the size of memory to allocate, while `new` automatically determines the size based on the type.
5. **Preferability:** In modern C++, `new`/`delete` are preferred for memory management, especially for objects, due to their type safety and automatic constructor/destructor invocation. However, `malloc`/`free` may still be used in specific situations, such as legacy code or interoperability with C libraries.

Overall, `new`/`delete` provide a more convenient and safer way to allocate and deallocate memory in C++, especially when dealing with objects and classes.

C++ Pseudocode:

```
#include <iostream>
using namespace std;

int main() {
    // Using malloc/free for dynamic memory allocation/deallocation
    int* ptr_malloc = (int*)malloc(sizeof(int)); // Allocates memory for an integer
    if (ptr_malloc == nullptr) {
        cout << "Memory allocation failed using malloc!" << endl;
        return 1;
    }
    *ptr_malloc = 10; // Assigning value to the allocated memory

    cout << "Using malloc/free: " << *ptr_malloc << endl;

    free(ptr_malloc); // Deallocates the memory

    // Using new/delete for dynamic memory allocation/deallocation
    int* ptr_new = new int; // Allocates memory for an integer
    if (ptr_new == nullptr) {
        cout << "Memory allocation failed using new!" << endl;
        return 1;
    }
    *ptr_new = 20; // Assigning value to the allocated memory

    cout << "Using new/delete: " << *ptr_new << endl;
```

```
    delete ptr_new; // Deallocates the memory

    return 0;
}
```

Node

Explanation:

In a linked list, a node is a fundamental building block. It contains two main components: data and a pointer to the next node in the sequence. The data part holds the actual information or value associated with the node, while the pointer part references the next node in the list. This pointer essentially establishes the connection between nodes, forming a chain-like structure.

Each node in a linked list is typically represented as a struct or a class in C++. The struct/class definition includes fields for data and a pointer to the next node. Through these pointers, nodes are linked together to form the entire linked list.

C++ Pseudocode:

```
#include <iostream> // Including the input-output stream library

using namespace std; // Using the standard namespace

// Node structure definition
struct Node { // Defining a struct named Node
    int data;       // Data part of the node, storing an integer
    Node* next;     // Pointer to the next node of type Node
};

int main() { // Main function
    // Creating nodes dynamically and storing their addresses in pointers
    Node* node1 = new Node();   // Creating a new node dynamically and assigning its addr
ess to node1
    Node* node2 = new Node();   // Creating another new node dynamically and assigning it
s address to node2

    // Assigning data to nodes
    node1->data = 10;   // Assigning the value 10 to the data member of node1
    node2->data = 20;   // Assigning the value 20 to the data member of node2

    // Linking nodes
    node1->next = node2;   // Setting the 'next' pointer of node1 to point to node2, link
ing them together

    // Traversing the linked list
    Node* current = node1;   // Initializing a pointer 'current' to point to the first no
de (node1) of the list
    while (current != nullptr) { // Looping until 'current' becomes nullptr, which indica
tes the end of the list
        // Printing the data of the current node
```

```
        std::cout << current->data << " "; // Using std::cout to print the data of the cu
rrent node, followed by a space

        // Moving to the next node
        current = current->next; // Updating 'current' to point to the next node in the l
ist
    }

    // Deallocating memory to prevent memory leaks
    delete node1; // Deleting the dynamically allocated memory for node1
    delete node2; // Deleting the dynamically allocated memory for node2

    return 0; // Indicating successful execution of the program
}
```

In this pseudocode, we define a `Node` struct to represent a node in the linked list. We create two nodes (`node1` and `node2`), assign data to them, and then link them together by setting the `next` pointer of `node1` to `node2`. Finally, we traverse the linked list, starting from `node1`, printing the data of each node. Afterward, we deallocate the memory allocated for the nodes using the `delete` operator to avoid memory leaks.

Linking nodes

C++ Pseudocode:

```
#include <iostream>
using namespace std;

// Define the structure of a node in the linked list
struct Node {
    int data;       // Data part of the node
    Node* next;     // Pointer to the next node in the list
};

int main() {
    // Create nodes for the linked list
    Node* head = new Node();   // Head node (first node in the list)
    Node* second = new Node(); // Second node
    Node* third = new Node();  // Third node

    // Assign data to each node
    head->data = 1;    // Data for the head node
    second->data = 2;  // Data for the second node
    third->data = 3;   // Data for the third node

    // Link the nodes together to form a linked list
    head->next = second; // Linking head node to second node
    second->next = third; // Linking second node to third node
    third->next = nullptr; // Marking the end of the list by setting the next pointer of
the last node to nullptr

    // Traversing the linked list and printing the data of each node
```

```
    Node* current = head; // Start from the head of the list
    while (current != nullptr) { // Loop until the current node is not nullptr
        cout << current->data << " "; // Print the data of the current node
        current = current->next; // Move to the next node
    }

    // Deallocate memory to prevent memory leaks
    delete head;
    delete second;
    delete third;

    return 0;
}
```

Explanation:

1. We define a `Node` struct which represents a node in the linked list. It contains two parts: `data`, which stores the value associated with the node, and `next`, a pointer to the next node in the list.
2. In the `main` function, we create three nodes `head`, `second`, and `third` dynamically using the `new` operator.
3. We assign values to the `data` member of each node.
4. To link the nodes together, we set the `next` pointer of each node to point to the next node in the list. This forms the chain-like structure of the linked list.
5. We traverse the linked list starting from the `head` node. In each iteration of the while loop, we print the `data` of the current node and then move to the next node using the `next` pointer.
6. Finally, we deallocate the memory allocated for the nodes using the `delete` operator to avoid memory leaks.

Traversing the linked list with the help of recursion

C++ Pseudocode:

```
#include <iostream>
using namespace std;

// Define the structure of a node in the linked list
struct Node {
    int data;       // Data part of the node
    Node* next;     // Pointer to the next node in the list
};

// Function to traverse the linked list recursively and print the data of each node
void traverseLinkedList(Node* current) {
    // Base case: If the current node is nullptr, return
    if (current == nullptr) {
        return;
    }

    // Print the data of the current node
    cout << current->data << " ";

    // Recursive call to traverse the next node in the list
```

```
    traverseLinkedList(current->next);
}

int main() {
    // Create nodes for the linked list
    Node* head = new Node();   // Head node (first node in the list)
    Node* second = new Node(); // Second node
    Node* third = new Node();  // Third node

    // Assign data to each node
    head->data = 1;    // Data for the head node
    second->data = 2;  // Data for the second node
    third->data = 3;   // Data for the third node

    // Link the nodes together to form a linked list
    head->next = second; // Linking head node to second node
    second->next = third; // Linking second node to third node
    third->next = nullptr; // Marking the end of the list by setting the next pointer of
the last node to nullptr

    // Traverse the linked list recursively and print the data of each node
    traverseLinkedList(head);

    // Deallocate memory to prevent memory leaks
    delete head;
    delete second;
    delete third;

    return 0;
}
```

Insertion and Deletion Operations in Linked Lists:

C++ Pseudocode:

```
#include <iostream> // Including the input-output stream library

using namespace std; // Using the standard namespace

// Define the structure of a node in the linked list
struct Node {
    int data;       // Data part of the node
    Node* next;     // Pointer to the next node in the list
};

// Function to insert a new node at the beginning of the linked list
void insertAtBeginning(Node*& head, int newData) {
    // Create a new node
    Node* newNode = new Node(); // Allocating memory for a new node
```

```
    newNode->data = newData; // Assigning data to the new node
    newNode->next = head; // Linking the new node to the current head
    // Update head to point to the new node
    head = newNode; // Updating the head pointer to the new node
}

// Function to insert a new node at the end of the linked list
void insertAtEnd(Node*& head, int newData) {
    // Create a new node
    Node* newNode = new Node(); // Allocating memory for a new node
    newNode->data = newData; // Assigning data to the new node
    newNode->next = nullptr; // Setting the next pointer of the new node to nullptr
    // If the list is empty, make the new node the head
    if (head == nullptr) { // Checking if the list is empty
        head = newNode; // Making the new node the head if the list is empty
        return; // Exiting the function
    }
    // Traverse to the last node
    Node* current = head; // Initializing a pointer to traverse the list
    while (current->next != nullptr) { // Looping until the last node is reached
        current = current->next; // Moving to the next node
    }
    // Link the last node to the new node
    current->next = newNode; // Linking the last node to the new node
}

// Function to delete a node with given data from the linked list
void deleteNode(Node*& head, int key) {
    // If the list is empty, return
    if (head == nullptr) { // Checking if the list is empty
        return; // Exiting the function
    }
    // If the key is found at the head node
    if (head->data == key) { // Checking if the key is found at the head node
        Node* temp = head; // Storing the address of the head node in a temporary pointer
        head = head->next; // Moving the head pointer to the next node
        delete temp; // Deallocating memory for the original head node
        return; // Exiting the function
    }
    // Traverse the list to find the node with the key
    Node* current = head; // Initializing a pointer to traverse the list
    Node* prev = nullptr; // Initializing a pointer to keep track of the previous node
    while (current != nullptr && current->data != key) { // Looping until the key is foun
d or the end of the list is reached
        prev = current; // Updating the previous pointer
        current = current->next; // Moving to the next node
    }
    // If the key is not found, return
    if (current == nullptr) { // Checking if the key is not found
        return; // Exiting the function
    }
```

```
    // Unlink the node with the key
    prev->next = current->next; // Updating the next pointer of the previous node to skip
the current node
    delete current; // Deallocating memory for the current node
}

// Function to display the linked list
void displayList(Node* head) {
    Node* current = head; // Initializing a pointer to traverse the list
    while (current != nullptr) { // Looping until the end of the list is reached
        cout << current->data << " "; // Printing the data of the current node
        current = current->next; // Moving to the next node
    }
    cout << endl; // Printing a newline character to move to the next line
}

int main() {
    Node* head = nullptr; // Initialize an empty linked list

    // Insert nodes at the beginning
    insertAtBeginning(head, 1); // Inserting a node with data 1 at the beginning
    insertAtBeginning(head, 2); // Inserting a node with data 2 at the beginning
    cout << "Linked list after inserting nodes at the beginning: ";
    displayList(head); // Displaying the linked list

    // Insert nodes at the end
    insertAtEnd(head, 3); // Inserting a node with data 3 at the end
    insertAtEnd(head, 4); // Inserting a node with data 4 at the end
    cout << "Linked list after inserting nodes at the end: ";
    displayList(head); // Displaying the linked list

    // Delete a node
    deleteNode(head, 2); // Deleting the node with data 2
    cout << "Linked list after deleting a node: ";
    displayList(head); // Displaying the linked list

    return 0; // Indicating successful termination of the program
}
```

Explanation:

- We first define the `Node` structure representing a node in the linked list.
- `insertAtBeginning` inserts a new node at the beginning of the list by creating a new node, linking it to the current head, and updating the head pointer.
- `insertAtEnd` inserts a new node at the end of the list by traversing to the last node, creating a new node, and linking it to the last node.
- `deleteNode` deletes a node with the given data by traversing the list to find the node, unlinking it from the list, and deallocating its memory.
- `displayList` traverses the list and prints the data of each node.

- In the `main` function, we demonstrate insertion and deletion operations by inserting nodes at the beginning and end and deleting a node from the list. Finally, we display the contents of the list.

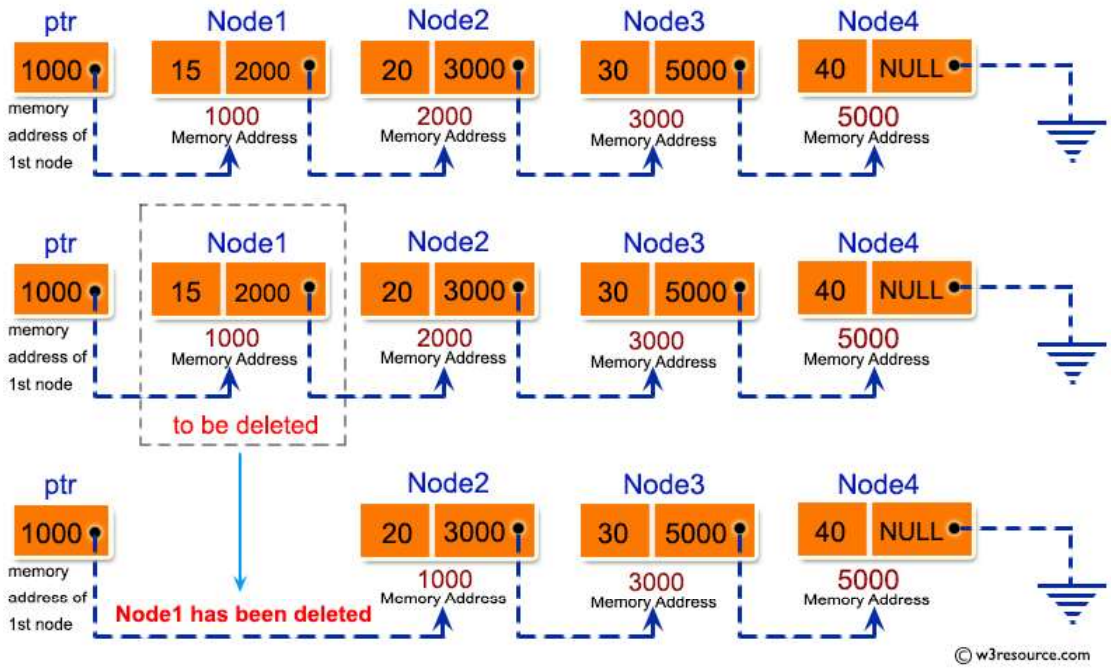
In deleteNode(), will the code fail without `temp`?

Using the `temp` variable ensures that we retain the address of the original head node before moving the `head` pointer, allowing us to safely deallocate the memory for the original head node later in the code:

```
// If the key is found at the head node
if (head->data == key) {
    Node* temp = head; // Storing the address of the head node in a temporary pointer
    head = head->next; // Moving the head pointer to the next node
    delete temp; // Deallocating memory for the original head node
    return; // Exiting the function
}
```

By using `temp`, we ensure that the memory allocated for the original head node is properly deallocated, preventing memory leakage and potential issues with resource management in the program. Therefore, while the code may technically execute without `temp`, it is best practice to include it to ensure proper memory management and avoid memory leaks.

A thorough visual depiction:



💡 **Role of & operator in function parameter**

The reason why the `&` is used in the function parameter list (`Node*& head`) but not within the function body is related to how C++ handles passing parameters to functions.

When you use `Node*& head` in the function parameter list, you're telling the function to accept a reference to a pointer to a `Node`. This means that any changes made to `head` within the function will affect the original pointer that was passed to the function.

Within the function body, you're working directly with the pointer itself, not with a reference to it. So, you don't need to use `&` when working with the pointer inside the function. You're already working with the original pointer, not a reference to it.

In summary, `Node*& head` in the parameter list allows you to modify the original pointer passed to the function, while within the function body, you work directly with the pointer itself.

Exercise

1. **Insertion and Deletion:**
   - Write a function to insert a new node with data 5 at the beginning of the linked list.
   - Implement a function to insert a new node with data 10 after the node containing data 3 in the linked list.
   - Create a function to delete all occurrences of a given key from the linked list.
   - Given a linked list which is sorted, how will you insert in sorted way?
2. **Traversal:**
   - Implement a function to recursively traverse the linked list and print the data of each node.
   - Write a function to calculate the sum of all elements in the linked list.
   - Implement a function to find the maximum value in the linked list.
3. **Combination of Operations:**
   - Write a function to reverse the linked list.
   - Implement a function to merge two sorted linked lists into a single sorted linked list.
   - Create a function to remove duplicates from an unsorted linked list.
4. **Edge Cases Handling:**
   - Write test cases to ensure proper handling of edge cases such as an empty list, a single-node list, or a list with only one distinct element.
   - Implement error-checking mechanisms to handle invalid inputs gracefully, such as deleting a node with data that does not exist in the list.

Deep dive topics

Constructor and Destructor

A constructor is a special member function in a class that is automatically called when an object of that class is created. Its primary purpose is to initialize the object's data members and set up the object's state. Constructors have the same name as the class and may have parameters. They are used to ensure that objects are initialized properly when they are created.

Here's an example of a simple constructor:

```
class MyClass {
public:
    int x;

    // Constructor
    MyClass(int value) {
        x = value;  // Initialize the data member
        cout << "Constructor called. Initialized x to " << x << endl;
    }
};

int main() {
    MyClass obj(5); // Constructor called with value 5
    return 0;
}
```

A destructor, on the other hand, is also a special member function in a class that is automatically called when an object goes out of scope or is explicitly deleted. Its primary purpose is to perform cleanup tasks, such as releasing resources or deallocating memory, associated with the object before it is destroyed. Destructors have the same name as the class preceded by a tilde (`~`), and they do not take any parameters.

Here's an example of a simple destructor:

```
class MyClass {
public:
    ~MyClass() {
        cout << "Destructor called. Object destroyed." << endl;
    }
};

int main() {
    MyClass obj; // Destructor called when obj goes out of scope
    return 0;
}
```

In summary, constructors are used to initialize objects when they are created, while destructors are used to perform cleanup tasks before objects are destroyed.

💡 **Type-Safety of malloc() and new()**

Type safety refers to the concept of ensuring that operations performed on data are compatible with the data types involved, thereby preventing type-related errors or inconsistencies. In the context of the statement you provided:

- **Type Safety with `new/delete`:**
  - `new` and `delete` operators in C++ are type-safe. When you use `new`, it allocates memory for an object of a specific type and calls the constructor of that type to initialize the object. Similarly, `delete` is aware of the type of object it is deleting and calls the destructor of that type to properly clean up resources.
  - Example:

    ```
    MyClass* obj = new MyClass(); // Type-safe allocation and construction
    delete obj; // Type-safe deletion and destruction
    ```

- **Type Safety with `malloc/free`:**
  - `malloc` and `free` functions in C are not type-safe. They operate on raw memory and do not have any knowledge of the type of data being allocated or deallocated. This means that they do not call constructors or destructors for objects.
  - Example:

    ```
    MyClass* obj = (MyClass*)malloc(sizeof(MyClass)); // Allocation without cons
    tructor call (not type-safe)
    free(obj); // Deallocation without destructor call (not type-safe)
    ```

In summary, the use of `new/delete` in C++ ensures type safety by handling memory allocation and deallocation in a way that is aware of the type of objects being created or destroyed, while `malloc/free` in C does not provide this level of type safety and operates solely on raw memory.

References

- Stack vs. Heap: Explaining the Difference | phoenixNAP KB
- Index of /class/archive/cs/cs106b/cs106b.1208/lectures (stanford.edu)
- (4) C Programming - YouTube
- Handy online cpp compiler: Online C++ Compiler (programiz.com)